

Disease Detection Agent Implementation

The objective of this sprint task is to create the core Disease Detection Agent that uses the pre-trained `crop_disease_mobilenetv2.keras` model to predict crop diseases from leaf images.

User Review Required

[!NOTE]

The model file is named `crop_disease_mobilenetv2.keras` and class names are in `class_names.npy` . I will use these instead of the requested `plant_disease_model.keras` which does not exist in the current directory.
The image expected input shape is `224x224x3` .

Open Questions

- Is it okay if the model is loaded once as a singleton or class attribute to prevent memory overflow on multiple invocations?
- Should the confidence levels thresholds (High: >90%, Medium: 70-90%, Low: <70%) be strictly greater/less than or inclusive (e.g. >= 90%)? I will default to `>=` for High and `>=` for Medium.

Proposed Changes

Disease Detection Agent

[NEW] `[disease_detection_agent.py]`

(file:///d:/agricare/agents/disease_detection_agent/disease_detection_agent.py)

This file will contain the `DiseaseDetectionAgent` class. It will:

- Load the MobileNetV2 model `crop_disease_mobilenetv2.keras` during initialization.
- Load the classes from `class_names.npy` .
- Implement a `predict(image_path)` method.

- The `predict` method will preprocess the image (resize to 224x224, scale/normalize correctly).
- Obtain the prediction probabilities, compute top-3 predictions.
- Format the return value as a JSON-like dictionary matching the requirements.

[NEW] `[init.py]`(file:///d:/agricare/agents/disease_detection_agent/init.py)

Initialize the python module.

[NEW] `[init.py]`(file:///d:/agricare/agents/init.py)

Initialize the python module for `agents` .

Unit Tests

[NEW] `[test_disease_detection_agent.py]`

(file:///d:/agricare/tests/test_disease_detection_agent.py)

This file will use Python's `unittest` framework to verify:

- The model correctly loads.
- Preprocessing works on dummy image data.
- The expected output structure (JSON dict) has all required fields: `disease` , `confidence` , `confidence_level` , `top_predictions` , and `warning` (when applicable).
- The `confidence_level` threshold logic works as expected.

Verification Plan

Automated Tests

- Run `pytest` or `python -m unittest tests/test_disease_detection_agent.py` to ensure all functionality is correct.

Manual Verification

- A small test script can be executed with a sample image to verify real output visually in the console.